Assignment 3: How Thread Count Affects a Shared-Memory Blocked Matrix Multiplication (OpenMP), with Optional Vectorisation

Hugo González Fernández
Big Data · Bachelor's Degree in Data Science and Engineering
Universidad de Las Palmas de Gran Canaria
Academic year 2026–2027 · Individual Assignment 3
Code repository: `https://github.com/hugogzfz/bd-assignment3-matmul`

November 2026

Abstract

Question. How does the number of threads affect the runtime, speedup and efficiency of a shared-memory dense `float64` matrix multiplication on a 4-core / 8-thread laptop CPU, and from which size does parallel execution pay off? **Method.** The blocked kernel of Assignment 2 (tile 96) is the sequential baseline; the parallel version splits the rows of C among OpenMP threads and runs the same kernel, so both perform bit-identical arithmetic on the same inputs. Thread counts 1–8 plus 16 (oversubscription), $n = 100$–4000, monotonic kernel timer, warm-up, 5 samples (3 at $n = 4000$), median and IQR, randomised order, a repeated pass, `ru_maxrss` memory, budgets of 120 s and 4 GiB, 1404 correctness checks and per-run validation; an optional extension compares scalar, SSE2 and AVX2 builds. **Results.** All 125 runs were correct and bit-identical to the baseline. Speedup is almost linear up to the 4 physical cores ($S(4) = 3.40$–3.95, efficiency 0.85–0.99) and saturates there: 8 threads change it by -15 to $+6\%$ and 16 threads give no consistent gain. Parallelism already paid off at $n = 100$ ($S(4) = 3.54$); a supplementary experiment added afterwards places the break-even between $n = 16$ (all parallel runs slower) and $n = 32$ ($S(4) = 2.56$). With one thread the OpenMP version matches the baseline within the between-process noise, and memory does not grow with the thread count. AVX2 is 3.9–5.1× faster than scalar code at equal thread count. **Scope.** One laptop, unpinned threads, ±10% variability between periods of the campaign and no hardware counters; conclusions apply to this kernel, timing boundary and machine.

1 Introduction and objective

Assignments 1 and 2 measured sequential dense matrix multiplication; the fastest sequential method found was a cache-blocked i-k-j kernel with tile $b = 96$ [2]. Its outer loop over rows of C is naturally parallel, because every row of $C = AB$ depends only on one row of A and on the whole of B. This study asks:

> *On a 4-core / 8-thread laptop CPU, how do runtime, speedup and efficiency of the blocked dense `float64` multiplication change with the number of OpenMP threads p, and from which matrix size n does parallel execution pay off?*

The factor changed is the thread count $p \in \{1, 2, 3, 4, 6, 8, 16\}$ (16 exceeds the 8 hardware threads: oversubscription), crossed with $n \in \{100, 250, 500, 1000, 1500, 2000, 3000, 4000\}$. The outcomes are time per multiplication, peak resident memory, speedup $S(p) = T_{seq}(n)/T_p(n)$ and efficiency $E(p) = S(p)/p$, all computed from **medians**. An optional, separate experiment compares scalar, SSE2 and AVX2 code generation of the same source at fixed thread counts.

Hypotheses (fixed before any measurement of the campaign).

H1 For $n \geq 1000$, $S(p)$ grows almost linearly up to the 4 physical cores, with $E(4) \geq 0.7$; it stays below 4 partly because the single-core turbo frequency is higher than the all-core one.

H2 Going from 4 to 8 threads (Hyper-Threading) adds little, between 0 and 30%, because the two hardware threads of a core share its floating-point units; $E(8) < E(4)$.

H3 With $p = 16$ (oversubscription) there is no gain over $p = 8$ and the time is slightly worse.

H4 For small n (≤ 250, kernel below about 1 ms) fork/join and scheduling costs dominate: $S(4) < 2$, low efficiency, and there is a size threshold below which parallel execution does not pay off.

H5 The parallel version with one thread takes the same time as the sequential baseline within $\pm 5\%$ for $n \geq 500$.

H6 Peak resident memory is $\approx 3 \cdot 8n^2$ bytes plus a constant and does not depend on p.

H7 (Optional extension) With one thread, the SIMD builds are faster than the scalar one (AVX2 > SSE2 > scalar); with many threads the relative SIMD gain shrinks somewhat.

2 Background

Blocking. Tiling the three loops so that a $b \times b$ working set of A, B and C is reused from cache reduces main-memory traffic [3, 4]; Assignment 2 found $b = 96$ best among the tested tiles on its machine [2]. **Shared-memory parallelism.** OpenMP creates a team of threads for a `parallel` region, which ends with an implicit barrier [5]. If a fraction s of the work is serial, Amdahl's argument bounds the speedup by $1/s$ [6]; fork/join, load imbalance and contention for shared resources (L3 cache, memory bandwidth, the floating-point units of a core shared by its two hardware threads under Hyper-Threading [7]) reduce efficiency further. The roofline model relates attainable throughput to arithmetic intensity and memory bandwidth [8]; blocked multiplication has high intensity, so it is expected to be compute-bound rather than bandwidth-bound. **Turbo frequency.** The i7-6820HQ has a base clock of 2.7 GHz and a maximum turbo clock of 3.6 GHz [9]; a single active core can run faster than four, so the ideal $S(p) = p$ is not attainable even for perfectly parallel work. **Vectorisation.** LLVM's loop vectoriser turns the contiguous inner j loop into packed SIMD instructions [10]; with FMA contraction disabled, each element of C still receives the same additions in the same order, so results stay bit-identical [11]. **Reporting.** Medians, IQRs and speedups from a stated baseline follow the recommendations of [12].

3 Methodology

3.1 Implementations

All code is C11 in one source file (`src/c/matmul3.c`); matrices are row-major `float64` arrays.

- **`seq` (equivalent baseline).** The blocked kernel of Assignment 2 [2] (tile $b = 96$, i-k-j order inside a tile), generalised only to work on a range of rows $[r_0, r_1)$ of C; the baseline calls it with $[0, n)$, which is exactly the Assignment 2 kernel. It never enters an OpenMP region and creates no thread (checked: 1 OS thread).
- **`omp` (parallel).** One `#pragma omp parallel num_threads(p)` region; thread t of q computes the contiguous rows $[\lfloor nt/q \rfloor, \lfloor n(t+1)/q \rfloor)$ of C with the same kernel. Each element of C is written by exactly one thread, A and B are only read, so there are no data races and no locks; the only synchronisation is the implicit final barrier. Rows are split evenly instead of distributing 96-row tiles, which would leave threads idle (e.g. $n = 1000$ has only 11 tile rows for 8 threads).
- **`ijk` (reference only).** The unchanged i-j-k kernel of Assignment 1 [1], run sequentially at $n = 500, 1000$ to place the study relative to the basic algorithm. It does not replace the equivalent baseline.

Because the summation order over k for every C_{ij} is identical in all three kernels and for every p, and FMA contraction is disabled, all methods must return *bit-identical* products; this is tested and checked in every benchmark run.

Build. Apple clang 14.0.0 with Homebrew `libomp` 23.1.5 (Apple clang ships without an OpenMP runtime):

```
clang -O2 -std=c11 -ffp-contract=off -Xpreprocessor -fopenmp
```

`-I$(brew -prefix libomp)/include -L$(brew -prefix libomp)/lib -lomp`.
No `-march=native` and no `-ffast-math`, as in Assignments 1–2. At `-O2` clang already vectorises the inner loop with 128-bit SSE2 instructions; this primary build is the "SSE2" build of the extension (byte-identical binary).

Thread control. Each run sets `OMP_NUM_THREADS`$= p$ and `OMP_DYNAMIC=false`; the runtime settings printed by `OMP_DISPLAY_ENV` are stored per run (`OMP_PROC_BIND=false`, `OMP_WAIT_POLICY=PASSIVE`, libomp defaults). The program reports, for the last call, the OpenMP team size, how many threads received rows, how many *distinct OS thread ids* executed the team (`pthread_threadid_np`) and the number of OS threads of the process (`task_threads`). The C code calls no numerical library; NumPy, used only for validation outside the measured processes, runs with `OPENBLAS_NUM_THREADS=1`. **Affinity:** macOS offers no thread-pinning interface (the Mach affinity policy is only a hint) and libomp does not support binding on macOS, so threads are *not pinned*; the OS scheduler places them. This is a limitation of the platform, recorded in `environment.json`.

3.2 Inputs and environment

The dense inputs are the Assignment 1/2 matrices: uniform $[0, 1)$ `float64` from NumPy's `default_rng` seeded with $[20261012, n, w]$ ($w = 0$ for A, 1 for B), written as little-endian binary files. `gen_inputs.py` regenerates them and verifies each SHA-256 against the digests of Assignment 2 (all 16 files identical). Table 1 summarises the machine. The laptop ran on mains power with all other applications closed, under `caffeinate -i`.

Table 1: Environment (from `results/environment.json`, recorded with `sysctl`, `sw_vers`, `clang -version`).

Item	Value
CPU	Intel Core i7-6820HQ CPU @ 2.70GHz
Cores / threads	4 physical, 8 logical (Hyper-Threading), 1 package
Caches	L1d 32 KiB and L2 256 KiB per core, L3 8 MiB shared, line 64 B
RAM	16 GiB
Machine / OS	MacBookPro13,3, macOS 12.7.6 (21H1320)
Compiler	Apple clang version 14.0.0 (clang-1400.0.29.202), libomp 23.1.5
Python (validation)	3.12.7, NumPy 2.4.2

3.3 Correctness checks before and during benchmarking

`test_correctness.py` runs `seq`, `ijk` and `omp` with $p \in \{1, 2, 3, 4, 7, 8, 16\}$ on: 1×1, 2×2 and 3×3 products with hand-computed answers, zero and identity matrices on both sides, an all-ones matrix, small-integer matrices (exact in `float64`), random matrices for $n \in \{5, 17, 64, 95, 96, 97, 100, 191, 250\}$ (below, at and above the tile size, not multiples of p), $n = 3$ with more threads than rows, seven tile sizes, the reported thread counts, and repeated calls into the same output buffer. Random cases are compared with NumPy [13] using the relative tolerance $2\gamma_n$, $\gamma_n = nu/(1-nu)$, $u = 2^{-53}$: each element of C is a length-n dot product of non-negative values, whose forward error is bounded by γ_n relative to the exact value for either summation order [11], so two correct orders differ by at most about $2\gamma_n$. Every product is also compared *bit for bit* with the `seq` result. The suite is run on the primary binary and on the three vectorisation builds (1404 $= 4 \times 351$ checks in total, all passed). During the campaign every process dumps its last product, which the driver validates against NumPy with $2\gamma_n$ and whose SHA-256 is compared with that of the sequential baseline of the same n.

3.4 Measurement protocol and budgets

- **Timer and boundary.** `clock_gettime(CLOCK_MONOTONIC)` around the multiplication call only. Inside: zero-filling C (required by the i-k-j update) and, for `omp`, the fork/join of the thread team. Outside: reading the input files, allocating and page-touching C, checksum, dump and validation. One process per configuration.
- **Repetitions.** 1 warm-up (recorded in the raw CSV, excluded from the statistics; it also lets the OpenMP runtime create its thread pool) and 5 measured samples; 3 samples at $n = 4000$ to keep the campaign within its time budget. Small products are batched so that a sequential sample lasts at

least about 0.2 s (batch sizes 607, 39 and 5 for $n = 100, 250, 500$); the reported value is time per multiplication. Statistics: median and interquartile range (IQR, linear-interpolation quartiles). Speedup and efficiency use medians; an IQR-based range $[Q_1(T_{seq})/Q_3(T_p),\ Q_3(T_{seq})/Q_1(T_p)]$ is reported as a rough uncertainty.

- **Order, pauses and temperature.** Within each n the configurations run in a seeded random order (the sequential baseline first, so that its SHA-256 is known), with a 3 s pause between processes. Before and after each process the driver stores the `CPU_Speed_Limit` reported by `pmset -g therm` (100 = no thermal limit). To estimate drift and between-process variability, the whole thread sweep at $n = 2000$ is **repeated in a second pass at the end** of the campaign.
- **Memory.** Peak resident set size of the process from `getrusage(RUSAGE_SELF).ru_maxrss`, which macOS reports in *bytes*; it includes inputs, output, code and thread stacks. The driver also polls the RSS with `ps` every 0.5 s to enforce the budget.
- **Budgets (declared before the campaign).** At most 120 s per timed sample and 4 GiB resident memory per process; a configuration whose $3 \cdot 8n^2$ array storage exceeds the memory budget is not attempted. Violations would be recorded as `timeout` / `memory_limit`, never with an invented time. The whole campaign was budgeted at 1–1.5 h; a 37 s pilot (disclosed here, not reported) was used only to estimate its duration.

3.5 Supplementary experiment: small sizes (added after the main campaign)

The main campaign showed a large speedup even at the smallest planned size ($n = 100$), so the size threshold of H4 was not located. As in Assignment 2, a supplementary experiment was therefore *added after seeing those results*: campaign `small` measures $n = 16, 32, 64$ with `seq` and `omp` $p \in \{1, 2, 4, 8\}$ under the same protocol (1 warm-up, 5 samples, batches of 147 965, 18 496 and 2 312 multiplications so that a sequential sample lasts about 0.2 s, randomised order, 3 s pauses, per-run validation). It ran separately, about two hours after the main campaign (22:19–22:20 UTC), with the laptop again idle and on mains power. The inputs come from the same generator and seed; their SHA-256 digests were added to the manifest (they did not exist in Assignment 2).

3.6 Optional vectorisation extension

The same source is compiled three times with the base flags above, changing only SIMD code generation: **scalar** (`-fno-vectorize -fno-slp-vectorize`), **SSE2** (default `-O2`, 128-bit, 2 doubles per instruction) and **AVX2** (`-mavx2`, 256-bit, 4 doubles). FMA stays disabled (`-ffp-contract=off`), so the three builds must give bit-identical results, which is tested. Evidence of vectorisation comes from the compiler (`-Rpass=loop-vectorize`) and from the disassembly of the kernel (`otool -tv`): Table 12. The campaign `vec` measures each build with `seq` and `omp` $p \in \{1, 2, 4, 8\}$ at $n = 1000$ and 2000 under the same protocol, so that SIMD effects (same p, different build) are separated from thread effects (same build, different p).

4 Results

All numbers below come from `results/runs.csv` and `results/raw_measurements.csv` through `scripts/analyze.py`. Times are medians of the measured samples, with the relative IQR (IQR/median) in parentheses.

4.1 Correctness, thread counts and campaign conditions

The campaign ran 110 processes in 21 minutes (start times 19:47:59–20:08:41 UTC, 12 Nov. 2026) and collected 542 measured samples. All 110 processes finished with status `ok`; no budget was reached, so the **largest attempted and largest feasible size is** $n = 4000$ (19.6 s per sequential multiplication, 367 MiB). Every product passed the NumPy check (largest relative error 8.5×10^{-15}, below $2\gamma_n$) and **all 110 products were bit-identical to the sequential baseline** of the same n, including every thread count, tile size and vector build. In all 88 `omp` processes the OpenMP team size and the number of distinct OS threads that executed it were equal to the requested p. `CPU_Speed_Limit` was 100 before and after every process (the OS never requested thermal throttling). The correctness suite (1404 checks: 351 on each of four binaries) reported no failure.

4.2 Runtime

Table 2 and Fig. 1 give the time per multiplication. The sequential baseline sustains 5.7–6.6 GFLOP/s ($2n^3/T$) for all sizes, so its time grows as n^3 (a factor 62 200 from $n = 100$ to 4000 for a factor 64 000 in n^3). The parallel version follows the same cubic trend shifted down by the speedup; four threads take 0.089 ms at $n = 100$ and 5.75 s at $n = 4000$. Variability within a process was small: relative IQRs were below 9% in 56 of the 60 configurations; the four larger ones (13–28%) are $p = 3$ at $n = 100$, $p = 8$ and 16 at $n = 250$, and $p = 8$ at $n = 2000$.

Table 2: Time per multiplication, median (relative IQR) of 5 measured samples (3 at $n = 4000$); campaigns `main` and `large`. Batches of 607, 39 and 5 multiplications for $n = 100, 250, 500$. "–": not planned. $p = 16$ is oversubscribed (8 hardware threads).

n	seq	p=1	p=2	p=3	p=4	p=6	p=8	p=16
100	314 µs (7.4%)	295 µs (0.57%)	151 µs (3.4%)	105 µs (16%)	88.8 µs (8.0%)	95.7 µs (4.3%)	86.3 µs (5.3%)	117 µs (2.1%)
250	4.73 ms (0.96%)	4.72 ms (0.76%)	2.48 ms (5.0%)	1.64 ms (1.5%)	1.33 ms (8.9%)	1.45 ms (0.44%)	1.56 ms (28%)	1.19 ms (24%)
500	40.0 ms (0.66%)	39.7 ms (2.7%)	19.9 ms (1.0%)	13.5 ms (0.89%)	10.6 ms (6.9%)	11.6 ms (2.6%)	10.5 ms (2.3%)	10.3 ms (3.3%)
1000	320 ms (0.40%)	319 ms (0.98%)	164 ms (3.3%)	108 ms (0.60%)	86.0 ms (2.8%)	90.1 ms (8.2%)	81.4 ms (1.6%)	84.8 ms (8.0%)
1500	1.18 s (5.0%)	1.10 s (1.6%)	553 ms (0.38%)	371 ms (0.87%)	298 ms (1.9%)	298 ms (3.2%)	281 ms (0.78%)	282 ms (1.7%)
2000	2.64 s (4.2%)	2.68 s (0.89%)	1.41 s (6.4%)	932 ms (8.5%)	755 ms (6.2%)	729 ms (2.5%)	744 ms (13%)	782 ms (3.0%)
3000	8.57 s (1.2%)	8.32 s (0.64%)	4.32 s (1.3%)	3.05 s (2.8%)	2.39 s (1.1%)	2.45 s (1.8%)	2.41 s (2.9%)	2.30 s (0.31%)
4000	19.6 s (0.52%)	19.5 s (0.66%)	–	–	5.75 s (2.1%)	–	5.55 s (1.8%)	–

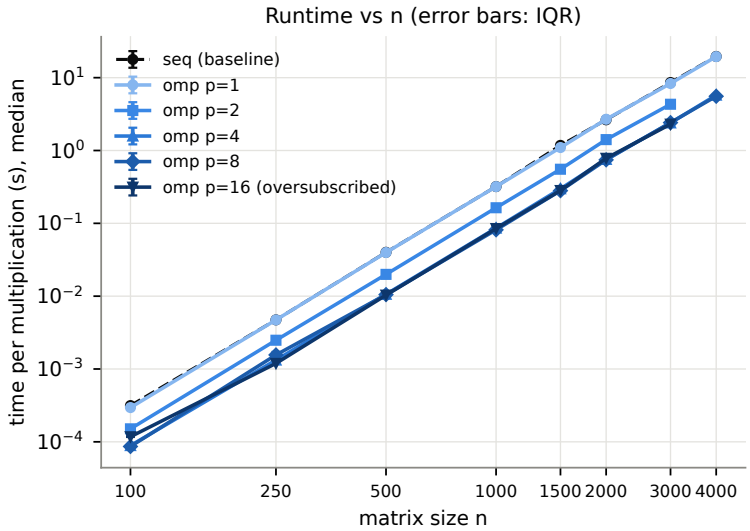


Figure 1: Median time per multiplication against n (log–log) for the sequential baseline and selected thread counts; error bars are the IQR (mostly hidden by the markers).

4.3 Speedup and efficiency

Table 3 and Figs. 2–3 show $S(p) = T_{seq}/T_p$ and $E(p) = S(p)/p$ from medians.

- **Up to the physical cores.** $S(2) = 1.87$–2.13, $S(3) = 2.81$–3.17 and $S(4) = 3.40$–3.95 for every n; $E(4) = 0.85$–0.99.
- **Beyond 4 threads.** Using the 8 hardware threads changes the speedup by a factor $S(8)/S(4) = 0.85$–1.06 (Table 4); for $n \geq 500$ the change is -1% to $+6\%$. $E(8) = 0.38$–0.52. $p = 6$ is never more than 2% better than the better of $p = 4$ and $p = 8$.
- **Oversubscription.** $S(16)/S(8) = 0.74$–1.31: no systematic gain; clear loss only at $n = 100$ (-26%); the $+31\%$ at $n = 250$ coincides with relative IQRs of 24–28% for $p = 8$ and $p = 16$.
- **Best configuration.** The highest median speedup per size is 3.52–4.19, reached with $p \in \{6, 8, 16\}$, but always within 7% of $S(4)$ except at $n = 250$ (11%, noisy).
- **Small sizes.** Even at $n = 100$, where one sequential multiplication takes 0.31 ms, four threads give $S = 3.54$ (IQR range 3.27–3.84). No tested size has $S(p) < 1$ for any $p \geq 2$ (Fig. 3).

Table 3: Speedup / efficiency, $S(p) = T_{seq}/T_p$ and $E(p) = S(p)/p$, from medians.

n	p=1	p=2	p=3	p=4	p=6	p=8	p=16
100	1.06 / 1.06	2.08 / 1.04	2.99 / 1.00	3.54 / 0.89	3.28 / 0.55	3.64 / 0.46	2.68 / 0.17
250	1.00 / 1.00	1.90 / 0.95	2.89 / 0.96	3.56 / 0.89	3.25 / 0.54	3.03 / 0.38	3.96 / 0.25
500	1.01 / 1.01	2.01 / 1.00	2.95 / 0.98	3.77 / 0.94	3.45 / 0.58	3.81 / 0.48	3.87 / 0.24
1000	1.00 / 1.00	1.95 / 0.98	2.97 / 0.99	3.72 / 0.93	3.55 / 0.59	3.93 / 0.49	3.77 / 0.24
1500	1.07 / 1.07	2.13 / 1.06	3.17 / 1.06	3.95 / 0.99	3.95 / 0.66	4.19 / 0.52	4.17 / 0.26
2000	0.98 / 0.98	1.87 / 0.94	2.83 / 0.94	3.50 / 0.87	3.62 / 0.60	3.55 / 0.44	3.38 / 0.21
3000	1.03 / 1.03	1.99 / 0.99	2.81 / 0.94	3.59 / 0.90	3.50 / 0.58	3.55 / 0.44	3.73 / 0.23
4000	1.00 / 1.00	–	–	3.40 / 0.85	–	3.52 / 0.44	–

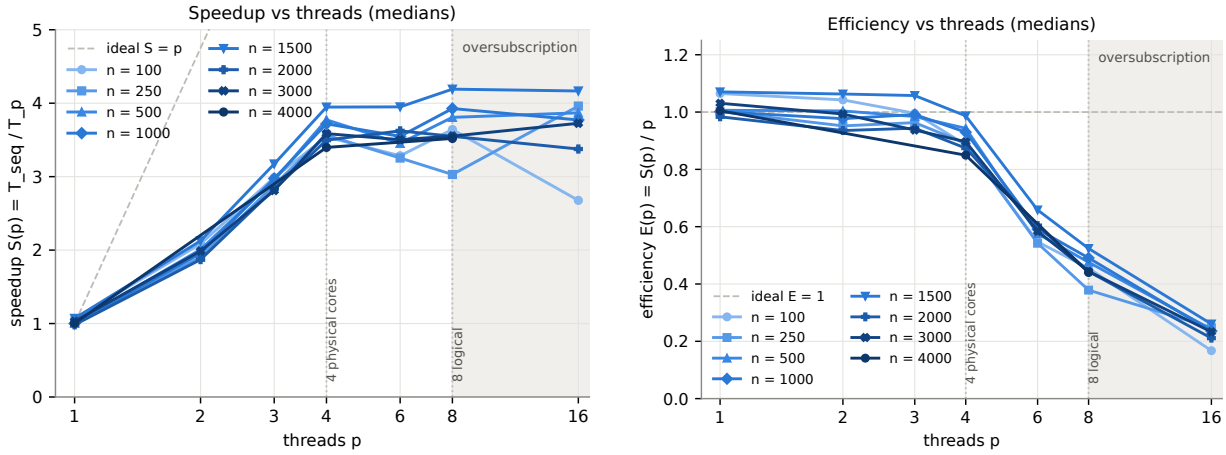


Figure 2: Speedup (left) and efficiency (right) against the number of threads, one line per n (darker = larger). Dashed: ideal. Dotted: 4 physical cores and 8 logical processors; shaded: oversubscription.

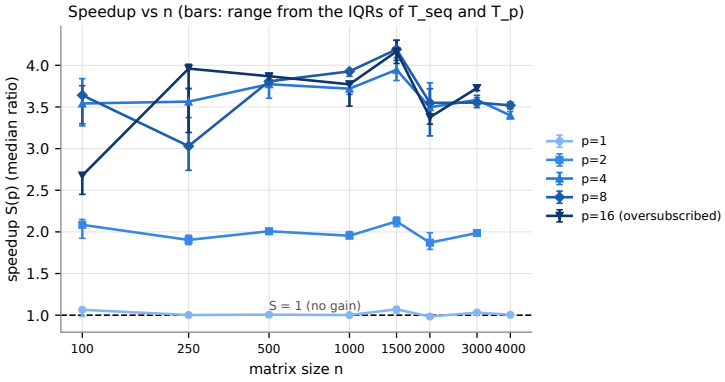


Figure 3: Speedup against n for selected p. Bars: rough range $[Q_1(T_{seq})/Q_3(T_p),\ Q_3(T_{seq})/Q_1(T_p)]$.

Table 4: Best median speedup per size, its IQR-based range and efficiency, and the effect of Hyper-Threading ($S(8)/S(4)$) and oversubscription ($S(16)/S(8)$).

n	best p	S (IQR range)	E	$S(8)/S(4)$	$S(16)/S(8)$
100	8	3.64 (3.30–3.76)	0.46	1.03	0.735
250	16	3.96 (3.19–4.01)	0.25	0.850	1.31
500	16	3.87 (3.75–3.90)	0.24	1.01	1.02
1000	8	3.93 (3.87–3.95)	0.49	1.06	0.960
1500	8	4.19 (4.06–4.30)	0.52	1.06	0.994
2000	6	3.62 (3.56–3.81)	0.60	1.02	0.951
3000	16	3.73 (3.70–3.75)	0.23	0.991	1.05
4000	8	3.52 (3.48–3.56)	0.44	1.04	–

Apparent superlinear values. $E > 1$ appears at $n = 1500$ ($E(2) = 1.06$, $E(3) = 1.06$), $n = 100$ ($E(2) = 1.04$) and for $p = 1$ at $n = 100$, 1500 and 3000 ($S(1) = 1.03$–1.07). They are not taken at face value:

in each case the parallel run with *one* thread—the same instructions on one core—is also faster than the sequential run (Table 5). Measured against the one-thread parallel run, $S_1(p) = T_{omp,1}/T_p$, the efficiencies are at most 1.00 for every n and p: at $n = 1500$, $S_1(2) = 1.99$, $S_1(3) = 2.96$, $S_1(4) = 3.69$; at $n = 100$, $S_1(2) = 1.96$. The cause is therefore a slower sequential process at those sizes (between-process variability, Section 4.6), not a cache effect of the parallel version; the sequential runs at $n = 100$ and $n = 1500$ also have the two largest relative IQRs among the sequential runs (7.4% and 5.0%).

4.4 Overhead of the parallel version with one thread

Table 5 compares `omp` with $p = 1$ against `seq`. The ratio is within ±5% for six of the eight sizes (0.970–1.017) and below 0.95 at $n = 100$ (0.939) and $n = 1500$ (0.934), where the *parallel* run is the faster one. No positive overhead of the OpenMP region larger than 2% is visible at any size.

Table 5: Parallel version with one thread vs. sequential baseline: median (IQR) and ratio of medians.

n	T_{seq} (IQR)	$T_{omp,1}$ (IQR)	$T_{omp,1}/T_{seq}$
100	314 μs (23.3 μs)	295 μs (1.70 μs)	0.939
250	4.73 ms (45.4 μs)	4.72 ms (35.7 μs)	0.998
500	40.0 ms (265 μs)	39.7 ms (1.06 ms)	0.994
1000	320 ms (1.29 ms)	319 ms (3.14 ms)	0.999
1500	1.18 s (58.7 ms)	1.10 s (17.2 ms)	0.934
2000	2.64 s (110 ms)	2.68 s (24.0 ms)	1.02
3000	8.57 s (102 ms)	8.32 s (53.6 ms)	0.970
4000	19.6 s (102 ms)	19.5 s (129 ms)	0.995

4.5 Memory

Peak resident memory (`ru_maxrss`) is the array storage $3 \cdot 8n^2$ plus a small constant: 0.7–0.8 MiB above it for the sequential process and 0.4–0.6 MiB more for any parallel process (Table 6, Fig. 4). Across thread counts at a fixed n, the parallel values differ by at most 0.34 MiB (at $n = 100$, between $p = 1$ and $p = 16$), i.e. memory does not grow with p beyond thread stacks and runtime structures.

Table 6: Peak resident set size (MiB, `getrusage` `ru_maxrss`, bytes on macOS) against the array storage $3 \cdot 8n^2$.

n	$3 \cdot 8n^2$ (MiB)	seq (MiB)	omp, all p (MiB)	max(omp)−seq (MiB)
100	0.229	1.02	1.30–1.64	0.621
250	1.43	2.21	2.50–2.79	0.570
500	5.72	6.51	6.79–7.09	0.574
1000	22.9	23.7	23.9–24.3	0.594
1500	51.5	52.3	52.6–52.8	0.551
2000	91.6	92.3	92.6–93.0	0.633
3000	206	207	207–207	0.602
4000	366	367	367–367	0.418

4.6 Variability between passes and within processes

The second pass at $n = 2000$, run ten minutes after the first at the end of the campaign, was **faster for every configuration, by 2.8–17.3%** (median −11%; Table 7, Fig. 5); it also had smaller IQRs (0.4–2.6% against 0.9–13%). Because sequential and parallel runs moved in the same direction, the speedups changed much less: $S(4)$ from 3.50 to 3.58, $S(8)$ from 3.55 to 3.78, $S(16)$ from 3.38 to 3.74. The same single-threaded computation at $n = 2000$ took 2.64 and 2.68 s in the first pass, 2.42 and 2.38 s in the second, 2.54 s in the tile sweep and 2.42–2.44 s in the vectorisation campaign: the slow interval is the first part of the campaign, not the end, so the difference is not progressive heating. Its cause could not be identified (no thermal limit was reported and no other application was open). Within single processes, the longest runs with $p \geq 3$ became slower from sample to sample: at $n = 3000$ the last sample took 4–8% longer than the warm-up and at $n = 4000$ 5–6% longer, while one-thread runs changed by less than 3% (Table 8). At $n = 2000$ (shorter runs) the changes have both signs (−7 to +8%).

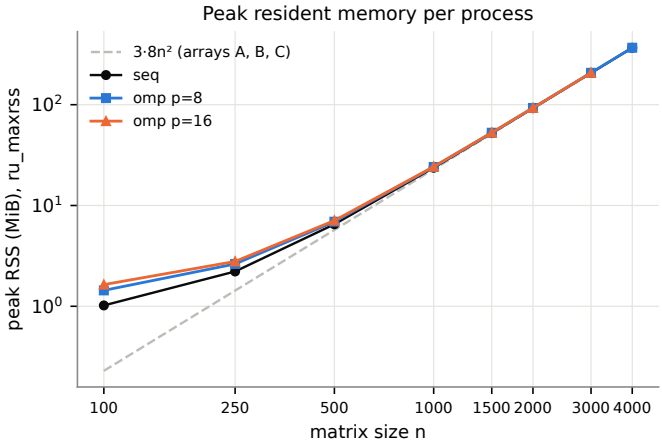


Figure 4: Peak resident memory per process against n; the parallel curves lie on top of the sequential one.

Table 7: Second pass at $n = 2000$: medians (relative IQR), difference pass 2 vs. pass 1, and speedups of each pass.

config	pass 1	pass 2	Δ (%)	S pass 1	S pass 2
seq	2.64 s (4.2%)	2.42 s (2.2%)	-8.4	–	–
p=1	2.68 s (0.89%)	2.38 s (2.2%)	-11.4	0.983	1.02
p=2	1.41 s (6.4%)	1.25 s (0.38%)	-11.7	1.87	1.94
p=3	932 ms (8.5%)	845 ms (0.46%)	-9.4	2.83	2.86
p=4	755 ms (6.2%)	675 ms (2.6%)	-10.6	3.50	3.58
p=6	729 ms (2.5%)	708 ms (1.9%)	-2.8	3.62	3.42
p=8	744 ms (13%)	640 ms (1.5%)	-13.9	3.55	3.78
p=16	782 ms (3.0%)	647 ms (1.1%)	-17.3	3.38	3.74

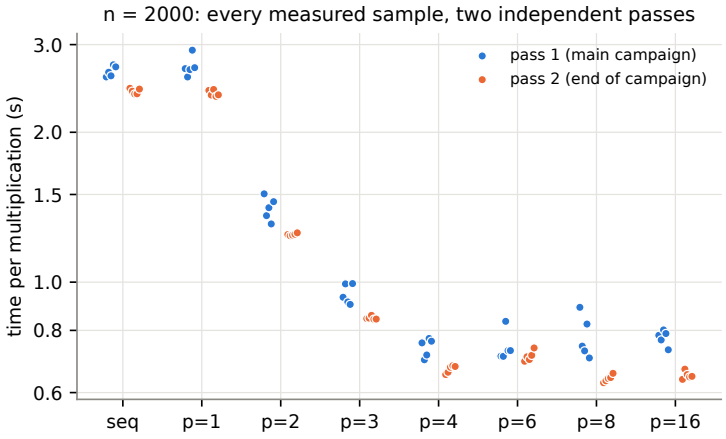


Figure 5: Every measured sample at $n = 2000$ in the first pass (main campaign) and in the second pass at the end of the campaign.

Table 8: Within-process drift: last measured sample relative to the warm-up sample of the same process (%), for the longest runs.

n	seq	p=1	p=2	p=3	p=4	p=6	p=8	p=16
2000	+1.2	-2.9	+1.9	-5.8	+8.4	+4.0	-0.0	-7.2
3000	-2.2	-1.6	+0.8	+4.2	+5.1	+4.5	+8.3	+7.2
4000	-1.0	+1.4	–	–	+6.2	–	+5.2	–

4.7 Reference comparisons

Basic algorithm. The unchanged Assignment 1 i-j-k kernel takes 140 ms at $n = 500$ and 1.86 s at $n = 1000$: 3.5× and 5.8× slower than the blocked baseline and 13× and 23× slower than the parallel version with 8 threads (Table 9). **Tile size.** The tile $b = 96$ was fixed before the campaign from Assignment 2. On this

CPU a short check at $n = 2000$ found $b = 64$ fastest sequentially (2.26 s vs. 2.54 s for $b = 96$) and $b = 32$ fastest with 4 threads (628 vs. 660 ms); $S(4)$ stayed between 3.54 and 3.86 for all five tiles (Table 10, Fig. 6).

Table 9: A1 i-j-k reference (sequential).

n	ijk (A1)	seq (blocked)	omp p=8	ijk/seq	ijk/omp8
500	140 ms (2.9%)	40.0 ms	10.5 ms	3.49	13.3
1000	1.86 s (1.6%)	320 ms	81.4 ms	5.81	22.8

Table 10: Tile-size check, $n = 2000$.

tile b	seq	omp p=4	$S(4)$
32	2.41 s (0.50%)	628 ms (0.78%)	3.83
64	2.26 s (1.0%)	638 ms (3.5%)	3.54
96	2.54 s (0.50%)	660 ms (0.66%)	3.86
128	2.46 s (2.0%)	686 ms (1.8%)	3.58
192	2.60 s (0.54%)	710 ms (1.2%)	3.66

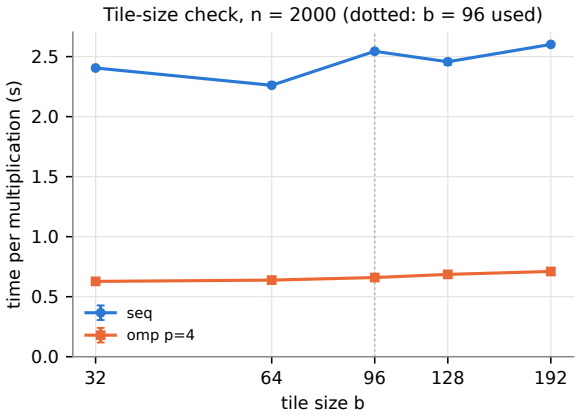


Figure 6: Time per multiplication against the tile size at $n = 2000$ (campaign `bsweep`); error bars are the IQR.

4.8 Supplementary experiment: where parallelism stops paying off

The 15 processes of the supplementary campaign were all correct, bit-identical to the baseline and without thermal limit. Table 11 and Fig. 7 show the result. At $n = 16$, where one sequential multiplication takes 1.47 μs, **every parallel configuration is slower than the baseline**: $S = 0.82$, 0.70, 0.53 and 0.40 for $p = 1, 2, 4, 8$, i.e. more threads make it worse. At $n = 32$ (9.6 μs) parallel execution already pays: $S(2) = 1.70$, $S(4) = 2.56$ (noisy: relative IQR 38%, IQR-based range 1.86–2.58), $S(8) = 2.06$; at $n = 64$ (70 μs) $S(4) = 2.95$. The break-even therefore lies between $n = 16$ and $n = 32$ for this machine and timing boundary. For $n \leq 64$, 8 threads are always slower than 4. Assuming the work itself divides perfectly, $T_p - T_{seq}/p$ at $n = 16$ estimates the cost of one parallel region: 0.33 μs with one thread, 1.4, 2.4 and 3.5 μs with 2, 4 and 8 threads.

Table 11: Supplementary campaign `small` (added after the main results): median time per multiplication (relative IQR for `seq`) and [speedup $S(p)$].

n	seq	p=1	p=2	p=4	p=8
16	1.47 μs (5.9%)	1.80 μs [0.82]	2.11 μs [0.70]	2.77 μs [0.53]	3.65 μs [0.40]
32	9.58 μs (0.77%)	10.0 μs [0.95]	5.63 μs [1.70]	3.74 μs [2.56]	4.65 μs [2.06]
64	69.6 μs (1.1%)	69.8 μs [1.00]	37.2 μs [1.87]	23.6 μs [2.95]	26.7 μs [2.60]

4.9 Optional extension: vectorisation

Evidence. Clang reports the inner j loop of the kernel (line 101) as "not vectorized" in the scalar build, "vectorized loop (vectorization width: 2, interleaved count: 2)" with SSE2 and "(vectorization width: 4, interleaved count: 4)" with AVX2. The disassembly of the kernel agrees (Table 12): the scalar build contains only scalar `mulsd`/`addsd`, the SSE2 build packed `mulpd`/`addpd` on 128-bit `xmm` registers, and the AVX2 build `vmulpd`/`vaddpd` on 256-bit `ymm` registers; no build contains FMA instructions. One scalar multiply and one scalar add remain in each build (in the vector builds, presumably the remainder iterations of the loop).

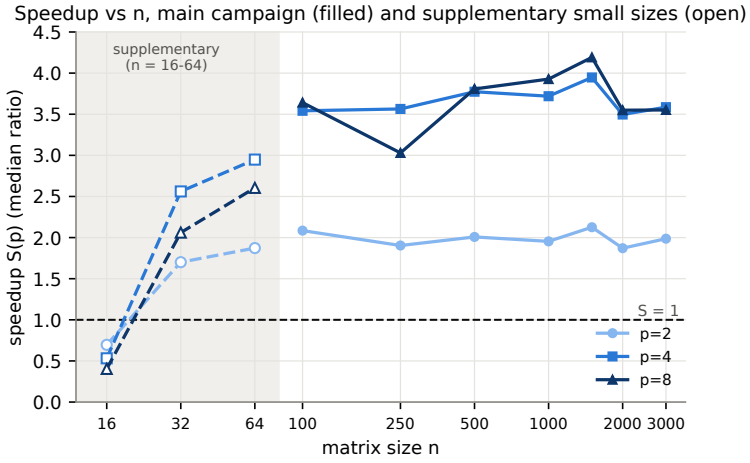


Figure 7: Speedup against n including the supplementary sizes $n = 16$–64 (open markers, separate later campaign; shaded) and the main campaign (filled). Below the horizontal line the parallel version is slower than the baseline.

Table 12: Vectorisation evidence: instruction counts in the disassembly of `mul_blocked_rows` (`otool -tv`) and clang remark for the inner loop.

build	extra flags	mulpd	addpd	mulsd	addsd	ymm	compiler remark (inner loop)
scalar	`-fno-vectorize -fno-slp-vectorize`	0	0	1	1	0	loop not vectorized
sse2	–	6	6	1	1	0	vectorized loop (vectorization width: 2, interleaved count: 2)
avx2	`-mavx2`	4	4	1	1	13	vectorized loop (vectorization width: 4, interleaved count: 4)

Performance. Table 13 and Fig. 8 cross the three builds with the thread counts. The bracketed factor is the *SIMD gain* $T_{scalar}(p)/T_{build}(p)$ at the same p and n, so it isolates code generation from thread count. SSE2 is 2.23–2.88× faster than scalar code and AVX2 3.94–5.08× faster, at every p. Within each build, the thread speedup at $p = 4$ is 3.09–3.56 (scalar), 3.61–3.89 (SSE2) and 3.38–3.56 (AVX2). Going from 4 to 8 threads raises the throughput of the scalar build at $n = 1000$ by 23% (235 to 192 ms) but slightly slows the AVX2 build (46.2 to 48.6 ms at $n = 1000$, 364 to 378 ms at $n = 2000$). At $n = 2000$ the AVX2 gain decreases from 4.40 (sequential) to 4.01 ($p = 8$); at $n = 1000$ it does not decrease monotonically (5.08 at $p = 4$). The fastest configuration, AVX2 with 4 threads, reaches 43.9 GFLOP/s at $n = 2000$, 14.9× the scalar sequential time. All 30 products were bit-identical to the baseline.

Table 13: Vectorisation × threads (campaign `vec`): median time and [SIMD gain $T_{scalar}(p)/T(p)$]. The SSE2 build is byte-identical to the primary binary.

n	build	seq	p=1	p=2	p=4	p=8
1000	scalar	726 ms [1.00]	714 ms [1.00]	371 ms [1.00]	235 ms [1.00]	192 ms [1.00]
1000	SSE2, 128-bit	317 ms [2.29]	306 ms [2.33]	161 ms [2.30]	81.6 ms [2.88]	80.2 ms [2.39]
1000	AVX2, 256-bit	165 ms [4.41]	168 ms [4.25]	84.8 ms [4.37]	46.2 ms [5.08]	48.6 ms [3.94]
2000	scalar	5.42 s [1.00]	5.45 s [1.00]	2.83 s [1.00]	1.52 s [1.00]	1.52 s [1.00]
2000	SSE2, 128-bit	2.42 s [2.24]	2.44 s [2.24]	1.25 s [2.26]	669 ms [2.28]	679 ms [2.23]
2000	AVX2, 256-bit	1.23 s [4.40]	1.23 s [4.42]	646 ms [4.37]	364 ms [4.18]	378 ms [4.01]

5 Discussion

5.1 Answer to the research question

On this 4-core / 8-thread laptop, the OpenMP version of the blocked kernel scales almost linearly up to the number of *physical* cores and then stops: $S(4) = 3.40$–3.95 ($E(4) = 0.85$–0.99) for every tested size, while 6, 8 or 16 threads change the speedup by at most a few percent for $n \geq 500$. In the main campaign parallel execution paid off at every size, down to $n = 100$ (0.31 ms per sequential product, $S(4) = 3.54$); the supplementary experiment located the threshold between $n = 16$ and $n = 32$: at $n = 16$ (1.5 μs of

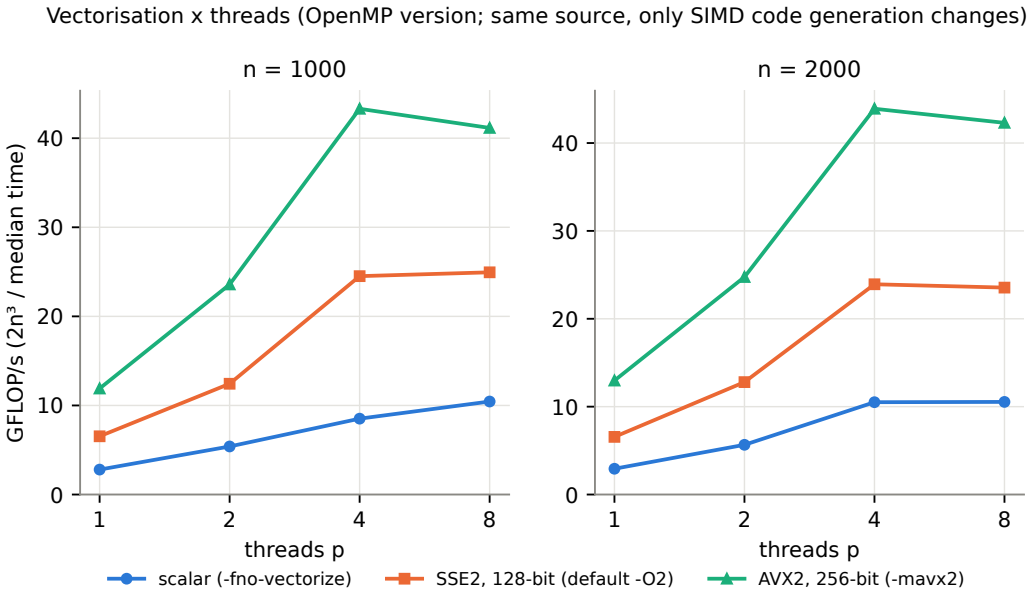


Figure 8: Throughput ($2n^3$/median time) of the parallel version for the three builds; vertical distance between curves = SIMD effect at fixed p, slope of each curve = thread effect for a fixed build.

work) every parallel configuration is slower than the baseline, at $n = 32$ (10 μs) four threads are 2.6× faster. The practical answer is therefore: parallelise once a product costs more than roughly ten microseconds of sequential work, and use as many threads as physical cores; more threads cost efficiency without a reliable gain.

5.2 Hypotheses against the evidence

- **H1 supported.** $E(4) \geq 0.85$ for all n, above the predicted 0.7, and $S(4) \leq 3.95$ for every size. Two observations support the frequency explanation without proving it (no frequency was measured): runs with $p \geq 3$ slow down by 4–8% during a process at $n = 3000$–4000 while single-thread runs do not (Table 8), as expected if the clock falls under sustained all-core load (the chip's maximum turbo clock, 3.6 GHz, is well above its 2.7 GHz base [9]); and the slowdown appears although `CPU_Speed_Limit` stays at 100, i.e. without OS-level throttling. Load imbalance cannot explain the gap: rows are split to within one row and every thread receives rows.
- **H2 supported.** $S(8)/S(4) = 0.85$–1.06 and $E(8) \leq 0.52 < E(4)$. The vectorisation campaign adds evidence for the mechanism [7]: Hyper-Threading raised the throughput of the scalar build by 23% at $n = 1000$, whose single thread leaves execution units idle, but not that of the SSE2 build (+2%) and slightly lowered that of the AVX2 build (−5%), whose single thread already keeps the floating-point units of a core busy.
- **H3 partly supported.** Oversubscription never gave a consistent gain over $p = 8$ ($S(16)/S(8) = 0.74$–1.31, with the high value inside a 24–28% IQR) and clearly hurt only at $n = 100$ (−26%), where each thread's share (6–7 rows, about 20 μs of work) is comparable to the cost of time-slicing 16 threads on 8 contexts and waiting at the barrier for the last one. For larger n the cost was not distinguishable from the noise.
- **H4 partly rejected.** The existence of a threshold was confirmed, but its position was wrong: the prediction of $S(4) < 2$ for $n \leq 250$ failed ($S(4) = 3.54$ and 3.56 at $n = 100$ and 250), and the threshold lies between $n = 16$ and $n = 32$ (supplementary experiment, added after the main results). The OpenMP runtime keeps its thread pool between parallel regions, so after the warm-up each region only has to restart the existing threads and synchronise at the final barrier. The estimated cost of a region, 1.4–3.5 μs for 2–8 threads, grows with p and is of the same order as the whole sequential product at $n = 16$ (1.47 μs), which explains why more threads are *worse* there; at $n = 32$ the 10 μs of work already amortise it. This conclusion is bound to the timing boundary: products were timed in batches after a warm-up; a single cold call, which includes the creation of the threads, was not measured separately.

- **H5 supported within the noise.** The one-thread parallel run differs from the baseline by −3% to +2% for six sizes; the two exceptions (−6 to −7%) go in the *opposite* direction of an overhead and coincide with the noisiest sequential runs. Any OpenMP overhead with one thread is smaller than the between-process variability (≈ ±5–10%, Section 4.6).
- **H6 supported.** Peak RSS = $3 \cdot 8n^2$ + 0.7–0.8 MiB (sequential) + 0.4–0.6 MiB with OpenMP, independent of p within 0.34 MiB. Each thread only allocates a stack and runtime bookkeeping; the matrices are shared, not copied.
- **H7 partly supported.** AVX2 > SSE2 > scalar at every p and n, as predicted. The SIMD gain shrank with threads only for AVX2 at $n = 2000$ (4.40 to 4.01); at $n = 1000$ it was not monotonic. The gains exceed the vector widths (2.2–2.9 for width 2, 3.9–5.1 for width 4). A plausible explanation, consistent with the compiler remarks, is that the vectoriser also interleaves the loop (2 or 4 independent vectors per iteration), so loop control, address arithmetic and load instructions are spread over 4–16 multiply–adds instead of one; without hardware counters this remains an explanation, not a measurement.

5.3 Why the speedup saturates at the physical cores

With FMA disabled, each update c_{ij} += $a_{ik}b_{kj}$ needs one multiply and one add. The blocked kernel keeps its tiles in the private L1/L2 caches [3, 4], so it is limited by those floating-point units and by load bandwidth, not by main memory: the sequential rate is almost constant (5.7–6.6 GFLOP/s) from $n = 100$ (all three matrices fit in cache) to $n = 4000$ (367 MiB), and the parallel rate is also flat (21–24 GFLOP/s with $p = 4$). In roofline terms [8] the kernel stays under the compute roof for every size, which is why the speedup does not depend on n and why adding hardware threads that share those units (H2) or software threads that share the contexts (H3) cannot help. The tile-size check points the same way: $S(4)$ was 3.54–3.86 for every tile from 32 to 192.

5.4 Limitations

- **One machine and one OS.** The results describe one Intel Skylake laptop under macOS; another CPU (more cores, Apple Silicon with performance and efficiency cores, different turbo policy) can change the shape of the curves.
- **No affinity, no counters, no frequency.** Threads could not be pinned, so their placement on cores and hardware threads was chosen by the scheduler and may differ between runs. Frequency, cache misses and port usage were not measured (`powermetrics` needs administrator rights); statements about turbo, caches and execution units are supported indirectly.
- **Between-process and between-period variability.** The same configuration differed by up to 17% between the two passes, much more than the within-process IQRs. Absolute times should be read with about ±10% uncertainty; speedups, whose two terms were measured minutes apart at the same size, were more stable (largest change 0.36 for $p = 16$). The cause of the slower first part of the campaign is unknown. With 5 samples per configuration (3 at $n = 4000$) quartiles are coarse.
- **Timing boundary.** Batches after a warm-up measure steady-state cost per product; the cost of the first parallel region (thread creation) is only present in the warm-up samples and is not analysed.
- **Tile size.** $b = 96$ was kept for comparability with Assignment 2, although $b = 64$ (sequential) and $b = 32$ (4 threads) were 5–11% faster here.

6 Conclusions

Within the tested scope (blocked i-k-j kernel, $b = 96$, `float64`, $n = 100$–4000, one i7-6820HQ laptop under macOS, steady-state timing):

- Row-partitioned OpenMP parallelisation of the Assignment 2 kernel is correct and race-free: all 125 measured products (110 main + 15 supplementary) and 1404 test products are bit-identical to the sequential baseline, and the reported thread counts match the real OS threads.
- The speedup grows almost linearly up to the 4 physical cores ($S(4) = 3.4$–4.0, $E(4) = 0.85$–0.99) and saturates there; Hyper-Threading ($p = 8$) and oversubscription ($p = 16$) add no reliable gain for this compute-bound kernel and halve or quarter the efficiency.

- Parallel execution stops paying off between $n = 16$ and $n = 32$ (about 1.5 vs. 10 μs of sequential work per product, in steady state): at $n = 16$ every thread count is slower than the baseline and more threads are worse; from $n = 32$ upwards 2–4 threads give $S = 1.7$–4.0.
- The OpenMP version with one thread has no overhead measurable above the ± 5–10% variability between processes, and memory does not grow with the number of threads.
- (Extension) SIMD code generation and threads multiply: AVX2 is 3.9–5.1× faster than scalar code at equal thread count, and AVX2 with 4 threads is 14.9× faster than the scalar sequential version at $n = 2000$ (43.9 GFLOP/s).

Unresolved: the cost of a single cold parallel call (thread creation), the exact break-even between $n = 16$ and 32, the cause of the period-to-period variability, and a direct measurement of clock frequency and cache behaviour, which would need hardware counters and administrator access.

7 Use of generative AI

In line with the course policy, the use of AI is disclosed: the tool was **Claude Code** (Anthropic). It was used for planning the experimental design and hypotheses, writing the C implementation (adapting the Assignment 2 kernel), the benchmark driver, the test, analysis, plotting and vectorisation-evidence scripts, the README, a first draft of this report and the oral-defence notes. It also searched for and checked the bibliographic data of the new references. The author approved the plan and the design decisions, supervised the execution of the measurement campaign on the author's own laptop, and reviewed the report and the oral-defence guide. The author remains responsible for understanding the code, validating the measurements and defending the conclusions.

Reproducibility materials

Repository: `https://github.com/hugogzfz/bd-assignment3-matmul` (code, Makefile, README with exact commands, raw CSV files, environment record, analysis scripts and figures). Reused material: the blocked kernel, input generator, test and driver structure of [2] (commit `1aab86d`) and the i-j-k kernel and inputs of [1] (commit `101af1c`); the inputs are regenerated and verified byte for byte with the SHA-256 digests in `data/manifest.json`.

References

[1] H. González Fernández, "Assignment 1: How runtime and memory scale in the basic triple-loop matrix multiplication in C, Java and Python," Big Data, Univ. Las Palmas de Gran Canaria, Oct. 2026. Code: `https://github.com/hugogzfz/bd-assignment1-matmul`, commit 101af1c.

[2] H. González Fernández, "Assignment 2: Optimised dense matrix multiplication and sparse matrices in C," Big Data, Univ. Las Palmas de Gran Canaria, Oct. 2026. Code: `https://github.com/hugogzfz/bd-assignment2-matmul`, commit 1aab86d.

[3] M. D. Lam, E. E. Rothberg and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, Santa Clara, CA, USA, 1991, pp. 63–74.

[4] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, pp. 1–25, 2008.

[5] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, Nov. 2021. [Online]. Available: `https://www.openmp.org/specifications/`

[6] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ, USA, Apr. 1967, pp. 483–485, doi: 10.1145/1465482.1465560.

[7] D. T. Marr, F. Binns, D. L. Hill, G. Hinton, D. A. Koufaty, J. A. Miller and M. Upton, "Hyper-Threading technology architecture and microarchitecture," *Intel Technology Journal*, vol. 6, no. 1, pp. 4–15, Feb. 2002.

[8] S. Williams, A. Waterman and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009, doi: 10.1145/1498765.1498785.

[9] "Intel Core i7-6820HQ notebook processor," Notebookcheck.net. [Online]. Available: `https://www.notebookcheck.net/Intel-Core-i7-6820HQ-Notebook-Processor.149416.0.html`. Accessed: Nov. 12, 2026.

[10] LLVM Project, "Auto-vectorization in LLVM," LLVM documentation. [Online]. Available: `https://llvm.org/docs/Vectorizers.html`. Accessed: Nov. 12, 2026.

[11] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[12] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC '15)*, Austin, TX, USA, 2015, Art. no. 73.

[13] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.